

Project Title	Rising Waters: Machine Learning Based Flood Prediction System		
Developer	N. Joshnavi devi	Team ID	[Team ID]
Date	2 July	Phase / Document	Project Demonstration

18. Demonstration of Proposed Features

The demonstration is designed to be run live, end-to-end, in the order below, so an evaluator sees the same request/response cycle described in the Data Flow Diagram (Section 5) actually happen in the browser rather than being described abstractly.

Feature	Demonstration Step
Application launch	Clone the repository (https://github.com/syedfatimanasreen-cmyk/raising-waters.git), start python app.py and open http://127.0.0.1:5000
Auto-Fill	Click Auto Fill to populate the form with realistic demo values
Manual input	Enter temperature, humidity, cloud cover, and rainfall figures manually
Prediction	Click Analyze Flood Risk and observe the returned probability and severity tag
Insights	Review the dynamic, human-readable explanation for the result
Protocols	Review the recommended safety protocols for the given severity tier
Edge case	Submit an invalid/missing field to confirm graceful error handling

The screenshot shows a VS Code editor with two files open: `README.md` and `app.py`. The `README.md` file contains project documentation, and the `app.py` file contains the Flask application code.

README.md Content:

```

1 Rising Waters: Flood Prediction System
2
3 Rising Waters is a Machine Learning-powered web application that analyzes meteorological data to predict
4 the likelihood of flood events. By examining factors such as annual rainfall, seasonal precipitation
5 patterns, humidity, and cloud cover, the system provides authorities and residents with early warnings to
6 aid in disaster response and evacuation planning.
7
8 ## 🌟 Features
9
10 - **Machine Learning Models**: Trained and evaluated multiple classification algorithms (Decision Tree,
11   Random Forest, K-Nearest Neighbors, XGBoost) on historical weather data to determine the most accurate
12   model for flood prediction.
13 - **Dynamic Frontend UI**: A premium, "glassmorphism" styled web interface built with HTML, CSS, and JS
14   that is fully responsive and interactive.
15 - **Instant Weather Analysis**: Input live or historical meteorological data to instantly receive a Flood
16   Probability Score.
17 - **Smart Data Auto-Fill**: A dynamic test-data generator that automatically populates the form with
18   realistic boundaries to demonstrate functionality instantly.
19 - **Contextual Insights**: The model doesn't just output a percentage; it provides dynamic, human-readable
20   insights explaining why the prediction was made (e.g., "Excessive annual precipitation detected") and
21   recommends actionable protocols based on severity.
22
23 ## 🛠 Technologies Used
24
25 - **Backend / Web Framework**: Python 3, Flask
26 - **Machine Learning**: Scikit-Learn, XGBoost, Pandas, NumPy
27 - **Frontend**: HTML5, Vanilla CSS3 (Glassmorphism design), JavaScript
28 - **Model Serialization**: Joblib
29
30 ## ⚙ Setup & Installation
31
32 **1. Clone the repository**
33 ```bash
34 git clone https://github.com/yourusername/raising-waters.git
35 cd raising-waters
36 ```

```

app.py Content:

```

1 from flask import Flask, render_template, request
2 import joblib
3 import pandas as pd
4 import os
5
6 app = Flask(__name__)
7
8 # Load trained model and scaler
9 MODEL_PATH = "models/flood_model.joblib"
10 SCALER_PATH = "models/scaler.joblib"
11
12 if not os.path.exists(MODEL_PATH) or not os.path.exists(SCALER_PATH):
13     raise FileNotFoundError("Model files not found! Please run train.py first.")
14
15 model = joblib.load(MODEL_PATH)
16 scaler = joblib.load(SCALER_PATH)
17
18
19 @app.route("/")
20 def home():
21     return render_template("index.html")
22
23
24 @app.route("/predict", methods=["POST"])
25 def predict():
26     try:
27         input_dict = {
28             "temp": float(request.form["temp"]),

```

notebook.ipynb ×

notebook.ipynb > import numpy as np

Generate + Code + Markdown | Run All Clear All Outputs | Outline ...

... Dataset loaded. Total rows: 115, Columns: 11

df.head()

[4]

...

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

df.tail()

[5]

...

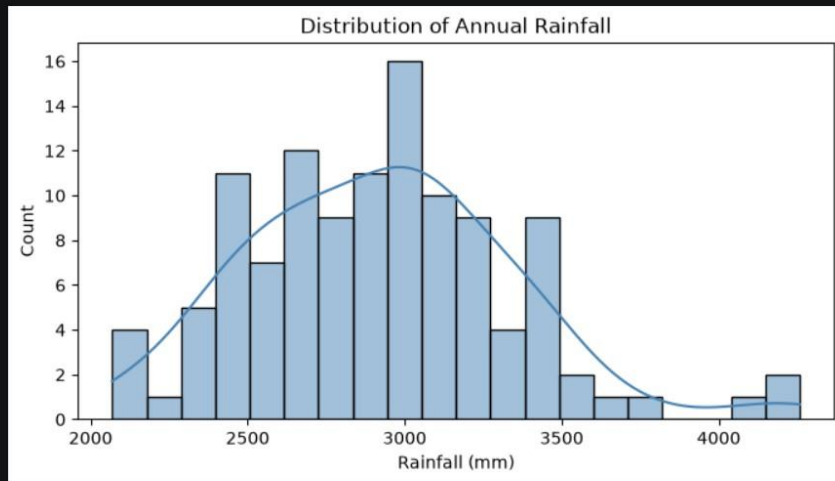
	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
110	28	71	30	3035.1	66.2	313.5	2209.1	446.3	262.833333	664.3	0
111	29	71	37	2151.1	18.3	287.4	1535.6	309.8	143.433333	335.0	0
112	30	74	42	3255.4	43.9	218.5	2561.2	431.8	347.566667	923.4	1
113	31	71	31	3046.4	14.9	364.5	2164.8	502.1	151.466667	203.4	0
114	28	71	34	2600.6	8.9	465.9	1514.7	611.1	187.866667	361.8	0

notebook.ipynb X

notebook.ipynb > import numpy as np

Generate + Code + Markdown Run All Clear All Outputs Outline ...

[12]



```
numeric_df = df.select_dtypes(include=[np.number])
plt.figure(figsize=(10, 8))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap of Weather Features and Flood Risk')
plt.show()
```

[8]

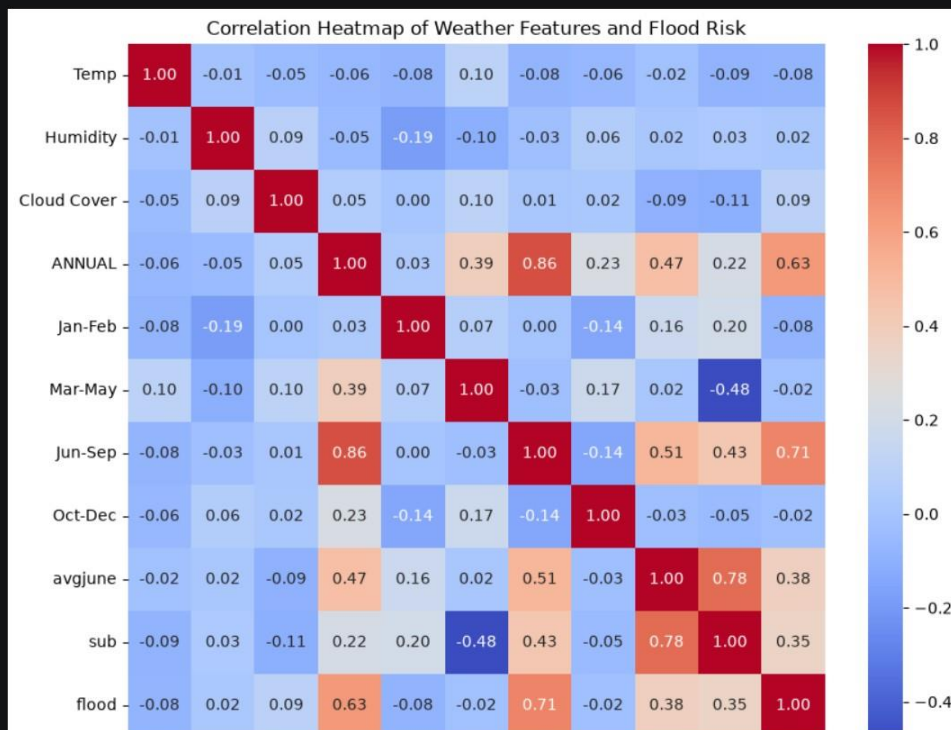
notebook.ipynb X

notebook.ipynb > import numpy as np

Generate + Code + Markdown Run All Clear All Outputs Outline ...

Select Kernel

[8]



The image shows a web browser window displaying the 'Rising Waters AI' application. The browser's address bar shows the URL '127.0.0.1:5000'. The application has a dark blue theme. The main heading is 'Rising Waters AI' with the subtitle 'Advanced Meteorological Flood Risk Assessment'. Below this is a 'Weather Parameters' section with an 'Auto Fill' button. The form contains several input fields for weather data, each with a placeholder value starting with 'e.g.'. At the bottom of the form is a large blue button labeled 'Analyze Flood Risk'.

Rising Waters AI
Advanced Meteorological Flood Risk Assessment

Weather Parameters Auto Fill

Temperature (°C) e.g. 28	Humidity (%) e.g. 75	Cloud Cover (%) e.g. 40
Annual Rainfall (mm) e.g. 3000	Jan-Feb Rainfall (mm) e.g. 50	Mar-May Rainfall (mm) e.g. 300
Jun-Sep Rainfall (mm) e.g. 2000	Oct-Dec Rainfall (mm) e.g. 500	Avg June Rainfall (mm) e.g. 200
Subdivision Code e.g. 500		

Analyze Flood Risk

The second screenshot shows the same application but with numerical values entered into the input fields:

Temperature (°C) 24	Humidity (%) 76.2	Cloud Cover (%) 73.7
Annual Rainfall (mm) 3650.4	Jan-Feb Rainfall (mm) 135.3	Mar-May Rainfall (mm) 731.6
Jun-Sep Rainfall (mm) 1373.1	Oct-Dec Rainfall (mm) 472.3	Avg June Rainfall (mm) 252.3
Subdivision Code 314.1		

Analyze Flood Risk



The Edge Case row is deliberately included alongside the happy-path features: showing that the system fails gracefully (a readable error message rather than a stack trace) is itself part of demonstrating the `/predict` route's `try/except` handling described in Section 13.